DOI:10.1145/2901919

Data-centric abstractions and execution strategies are needed to exploit parallelism in large-scale graph analytics.

BY ANDREW LENHARTH, DONALD NGUYEN, AND KESHAV PINGALI

Parallel Graph Analytics

THE STUDY OF graphs began in the 18th century with Euler's work on the "Bridges of Königsberg" problem, but it is only since the creation of the Web two decades ago that large-scale parallel computers have been used to study graph properties. Today, this field, known as "parallel graph analytics," touches all our lives, even if we are not aware of it. When we use a Web search engine or get a friend recommendation on Facebook, graph analytics is at work behind the scenes. Companies like Amazon and Netflix use parallel graph analytics to find patterns of purchases by their customers, enabling them to make targeted product recommendations. And intelligence agencies use parallel graph analytics to find key players, called "centralities," in terrorist networks.

Using computers to study graph properties is not new; 40 years ago, in 1976, mathematicians Kenneth Appel and Wolfgang Haken ran a FORTRAN program for 1,200 hours to prove the four-color theorem for planar graphs.[a] Today, we are interested in studying not only the mathematical properties of different kinds of graphs (such as planar graphs) but also in computing and exploiting properties of particular graphs that arise from big datasets. There are three main challenges in performing computations on these graphs—large graph sizes, diversity in graph structure, and the complex patterns of parallelism in graph analytics applications. Most current parallel abstractions and implementations fail to provide solutions for at least one of them.

Large graph sizes. In today's era of big data, parallel graph analytics systems must deal with graphs with billions of nodes and trillions of edges; for example, Web search engines are based on the indexable Web graph,[19] which has more than 50 billion nodes, representing webpages, and one trillion edges, representing hyperlinks between pages. The Facebook graph has approximately one billion nodes, representing Facebook users, and 200 billion edges, representing friendships between users.[36] Parallel computing is essential for analyzing properties of graphs of such size in reasonable time.

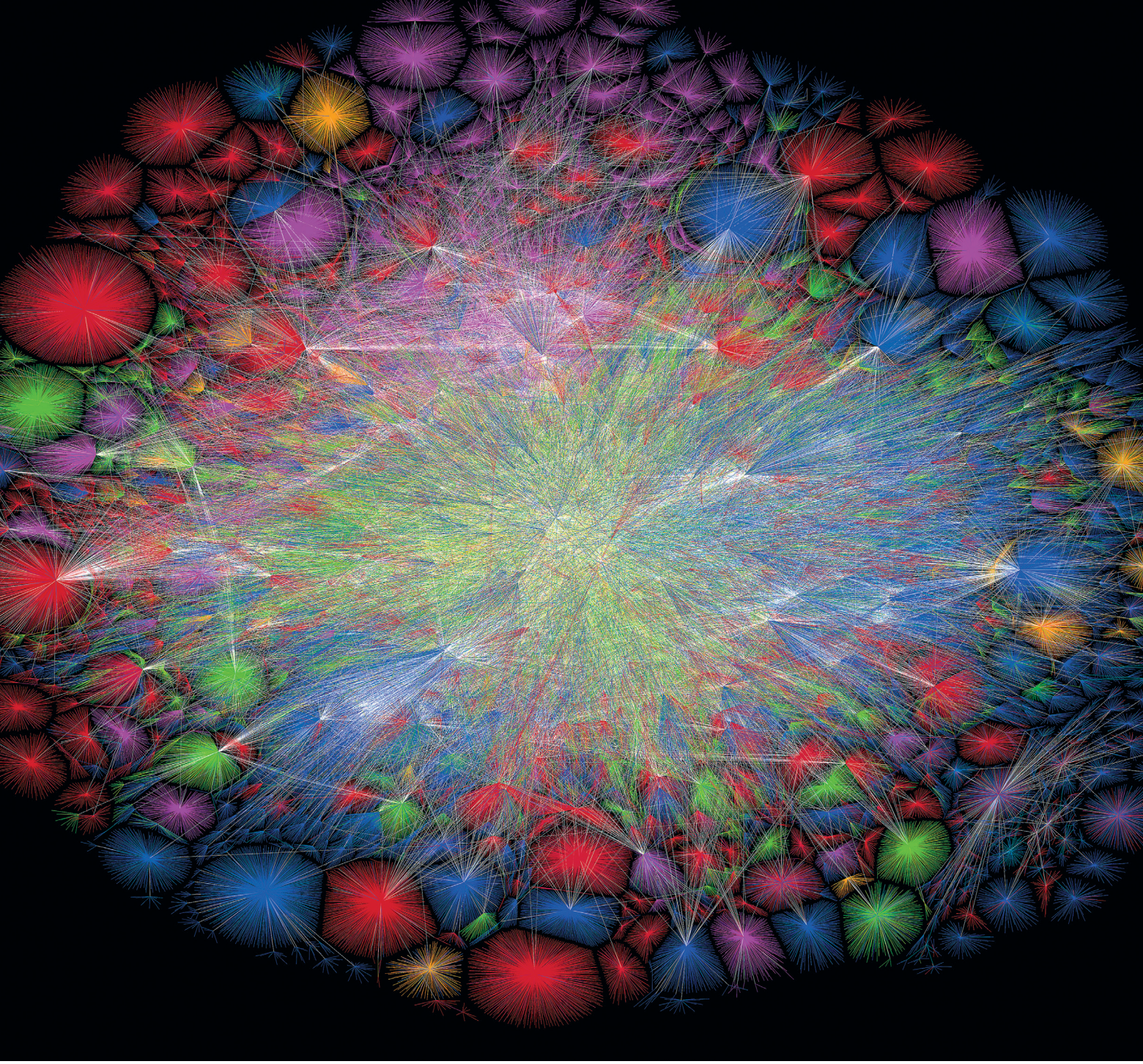
Diversity in graph structure. Graph structure is very diverse and has a major impact on the performance of graph analytics algorithms.

Planar graphs and graphs (such as road networks) have the property that the number of edges connected to a node, or the "degree" of that node, is fairly uniform across all nodes and is a small number that does not depend on the size of the graph. As a consequence, the average length of the shortest path between pairs of nodes, or the "average diameter of the graph," increases rapidly as we consider graphs of increasing size.

» key insights

- **Parallelism in graph algorithms is qualitatively different from parallelism in high-performance computing algorithms.**
- **Data-centric abstractions are key to understanding parallelism in graph analytics applications.**
- **Different parallel execution strategies are needed for different types of graphs.**

a A graph is planar if it can be drawn on a sheet of paper so its edges do not cross each other.



Visualization of the routing paths of the Internet.

IMAGE BY BARRETT LYON/THE OPTE PROJECT

Studies have shown social network graphs like the Web and Facebook graphs have a very different structure from planar graphs; although the average degree of a node is small, the average diameter of randomly generated social network graphs grows only as the logarithm of the number of nodes.[7,21] To understand how this can happen, consider an airline route map. Most airlines organize flights on the hub-and-spoke model; a small number of cities are chosen as hubs and connected to many other cities by direct flights, but most cities have only a small number of flights, usually only to one hub. Nevertheless, you can travel between most pairs of cities by taking just three flights—a direct flight to the hub that serves your starting city, a flight from there to a hub that serves your destination, and then a flight on to your destination.

Like airline route maps, social network graphs have a "lumpy" node structure, known formally as a "power-law structure," in which a few richly connected hubs can be exploited to traverse the network in a small number of hops; for example, the average diameter of the Facebook graph is approximately five, even though the graph has more than one billion nodes.[36]

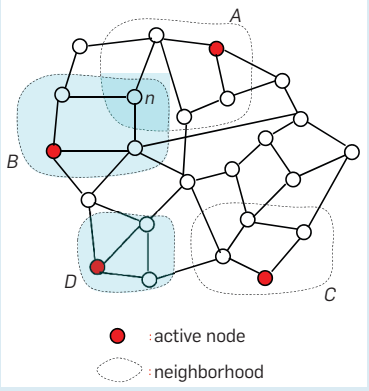
A third category of graphs, called "random graphs" or "Erdös-Rényi graphs," is useful for studying certain combinatorics problems. One way to construct random graphs is to start with some number of nodes and then add edges incrementally between randomly selected pairs of nodes. In contrast, power-law graphs arise from a "rich get richer" generative model in which the selection is biased in favor of nodes with higher degrees.

For many analytics problems, different algorithms must be used for uniform-degree, random, and power-law graphs to obtain good parallel performance, as we show here.

Need for new parallel abstractions. Most existing abstractions and implementations for parallel computing were developed for computational science applications in which the main parallelism pattern is data parallelism; an example of a data-parallel operation is "map," which applies a function to each element of a set, producing a new set. Systems like Hadoop enable programmers to express and exploit data parallelism without having to write low-level parallel code.

Some graph analytics algorithms have data parallelism, but others exhibit a more complex parallelism pattern called "amorphous" data-parallelism,[29] described later in this article. Unlike in data-parallel programs, tasks in amorphous data-parallel programs may or may not be able to run in parallel, depending on the graph structure and values known only at runtime. To exploit this parallelism pattern, systems need to find opportunities for parallel execution while executing the program in parallel. As a consequence, abstractions and implementations for data-parallelism are not adequate for many parallel graph analytics algorithms.

Figure 1. Operator formulation.



Landscape of Graph Analytics Algorithms

To understand the patterns of parallelism in graph analytics algorithms, it is convenient to use concepts from the "operator formulation of algorithms,"[29] a data-centric abstraction of algorithms shown pictorially in Figure 1. To illustrate these concepts, we use the Dijkstra and Bellman-Ford algorithms for the single-source shortest-path problem (SSSP).[5] Given a weighted, undirected graph $G = (V,E,w)$, where V is the set of nodes, E is the set of edges, and w is a map from edges to positive edge weights, the SSSP problem is to compute the length of the shortest path from a given source node s to every other node. Both algorithms are described in detail in the next section.

The operator formulation has a local view and a global view of algorithms, as summarized in Figure 2.

The local view is described by an operator, which is a graph update rule applied to an active node in the graph; some algorithms have active edges, but, here, we refer only to active nodes. Each operator application, called an "activity," reads and writes a small region of the graph, called the "neighborhood" of that activity, around the active node. In Dijkstra's algorithm, the operator, called the "relaxation" operator, uses the label of the active node to update the labels of its neighbors. Figure 1 shows active nodes as filled dots and neighborhoods as clouds surrounding active nodes for a generic algorithm. An active node becomes inactive when the activity is completed.

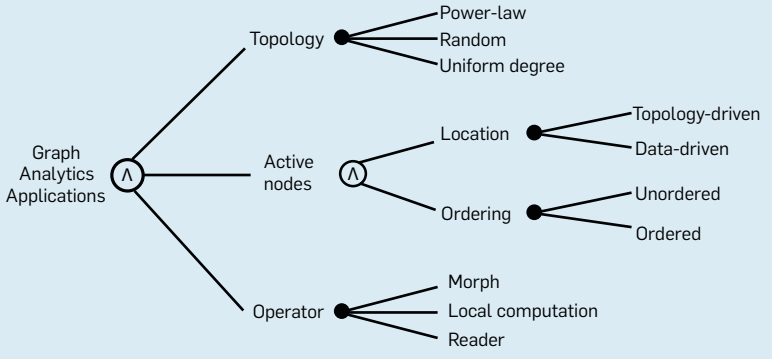
In general, operators can modify the graph structure of the neighborhood by adding and removing nodes and edges; they are called "morph" operators. In most graph analytics applications, operators update only labels on nodes and edges without changing the graph structure. These operators are called "local computation" operators; a "pull-style" operator reads the labels of nodes in its neighborhood and writes to the label of its active node, while a "push-style" operator reads the label of the active node and writes the labels of other nodes in its neighborhood. Dijkstra's algorithm uses a push-style operator. In algorithms that operate on several data structures, some data structures may be read-only, in which case the operator is a "reader" for those data structures.

Neighborhoods can be distinct from the set of immediate neighbors of an active node, and can, in principle, encompass the entire graph, although usually they are small regions of the graph surrounding the active node. Neighborhoods of different activities can overlap; in Figure 1, node n is contained in the neighborhoods of both activities A and B. In a parallel implementation, the semantics of reads and writes to such overlapping regions, or the "memory model," must be specified carefully.

The global view of the algorithm is described by the locations of active nodes in the graph and the order in which they must appear to have been ved by the implementation.

In "topology-driven" algorithms, the locations of active nodes are determined by the graph structure. These algorithms make a number of sweeps over the graph; in each such sweep, all nodes are initially active,[b] and the

Figure 2. TAO analysis of graph applications.



b In some algorithms, a subset of nodes (such as interior nodes in a mesh) are initially active in each sweep, but this subset is determined by the graph topology.

sweeps terminate when some global quiescence condition is reached; the Bellman-Ford algorithm is an example. On the other hand, in data-driven algorithms, there is an initial set of active nodes, and the execution of activities may cause other nodes to become active on the fly. These algorithms do not have any algorithmic notion of sweeps, terminating when there are no more active nodes in the graph. Dijkstra's algorithm is an example; only the source node is active initially and other nodes become active as a side effect of graph updates.

The second dimension of the global view of algorithms is "ordering." In "unordered" algorithms, any order of processing active nodes is semantically correct; each sweep of the Bellman-Ford algorithm is an example. Some orders may be more efficient than others, so unordered algorithms sometimes assign soft priorities to activities, but these priorities are only suggestions to the runtime system, and priority inversions are permitted in the execution. In contrast, "ordered" algorithms requiring active nodes appear to have been processed in a specific order; Dijkstra's algorithm is an example. This order is specified by assigning priorities to active nodes, and the implementation is required to process active nodes so they appear to have been scheduled for execution in strict priority order, from earliest to latest.

Like SSSP, many graph analytics problems can be solved by both topology-driven and data-driven algorithms, but each type of algorithm makes different trade-offs. Topology-driven algorithms are easier to implement because graph representations usually store the nodes of the graph in an array or a list, so iteration over active nodes can be implemented simply as iteration over this data structure. In contrast, data-driven algorithms must use a workset (or multiset) to hold the active nodes. Sequential implementations usually use lists and priority queues to implement worksets for unordered and ordered algorithms, respectively. Nguyen et al.[27] described a scalable implementation of a parallel workset supporting soft priorities.

Data-driven algorithms can be more work efficient. Topology-driven algorithms visit nodes even if there is no work to do there, whereas data-driven algorithms can focus on regions of the graph where useful work needs to be done, making them asymptotically faster than their topology-driven counterparts for many problems; for example, the asymptotic complexity of the Bellman-Ford algorithm is $O(|E||V|)$, whereas the complexity of Dijkstra's algorithm is $O(|E| \log(|V|)$.

Unlike in data-parallel programs, tasks in amorphous data-parallel programs may or may not be able to run in parallel, depending on the graph structure and values known only at runtime.

Figure 2 is a graphical presentation of these categories of graph applications. We call it TAO analysis for its three main dimensions—Topology of the input graph, Active nodes location and ordering, and Operator. There is no explicit notion of parallelism in TAO analysis or a distinction between sequential and parallel algorithms.

These concepts map directly to programming constructs in object-oriented languages. Graphs are implemented using abstract data types that support a common API but represent the graph in memory in different schemes. Iteration over active nodes can be expressed succinctly by unordered and ordered set iterators that permit new elements to be added to the set during the iteration.[29] For topology-driven algorithms, this iterator is nested within an ordinary (ordered) loop that iterates until some convergence criterion is reached. The body of the iterator is an implementation of the operator and makes calls to a graph API for accessing graph elements. Some algorithms have multiple phases, and each phase may use a different operator; other algorithms may require the interleaved execution of several operators.

Illustrative Examples

We illustrate these ideas through algorithms for SSSP and collaborative filtering.

There are many algorithms for SSSP, including chaotic relaxation, Dijkstra, delta-stepping, and Bellman-Ford.[5,25] In the literature, they appear to be unrelated to each other, but TAO analysis reveals their similarities and differences. Each node v is given a distance label $dist(v)$ that maintains the length of the shortest known path to that node from the source. This field is initialized to 0 at the source s and ∞ at all other nodes. The relaxation operator in SSSP algorithms is defined as follows: if (u,v) is an edge and $dist(u) + w(u,v) < dist(v)$,

the value of *dist*(*v*) is updated to *dist*(*u*) + *w*(*u*, *v*). An active node is relaxed by applying this operator to all the edges connected to it. This is a push-style operator, though it is also possible to implement a pull-style operator. Each relaxation may lower the *dist* label of some node, and when no further lowering can be performed anywhere in the graph, the resulting node labels are the shortest distances from the source to the nodes, regardless of the order in which the relaxations were performed. Breadth-first numbering of a graph is a special case of SSSP in which all edge labels are 1.

Differences between SSSP algorithms arise in the active nodes dimension of TAO analysis.

"Chaotic relaxation"[3] is a data-driven, unordered algorithm. The workset initially contains only the source node (for the push-style operator), and active nodes are chosen randomly by the algorithm for execution from the workset. This algorithm exposes a lot of parallelism for most input graphs, but the number of relaxations may be exponential in the graph size, so the algorithm is not work efficient.

Dijkstra's algorithm is a data-driven, ordered algorithm with a push-style operator. The workset initially contains only the source node. Active nodes are scheduled in strictly non-decreasing order of node labels, using a priority queue in which the key is the node label. This algorithm is work efficient but may not expose much parallelism.

"Asynchronous delta-stepping"[c] is a compromise between the extremes of chaotic relaxation and Dijkstra's algorithm and can be viewed as controlled chaotic relaxation. The workset is implemented as a sequence of buckets, and an active node with label *d* is mapped to the bucket at position d / Δ, where Δ is a user-defined parameter. Nodes within a given bucket can be relaxed in any order, but buckets are processed in strictly increasing order. If Δ is 1, the algorithm behaves like Dijkstra's algorithm, and if $\Delta = \infty$, the algorithm behaves like chaotic relaxation.

For many analytics problems, different algorithms must be used for uniform-degree, random, and power-law graphs to obtain good parallel performance.

In social network graphs, a small value of Δ is preferable, as this choice promotes work efficiency without choking off parallelism;[d] for uniform-degree graphs, a larger-value Δ must be used to expose parallelism.

Bellman-Ford is a topology-driven algorithm that makes $O(|V|)$ unordered sweeps over the graph. Most implementations use a push-based operator.

In conventional Bellman-Ford, updates to a node label in a sweep are available immediately to other activities in that sweep; such sweeps are Gauss-Seidel iterations in the linear algebra sense. "Jacobi-style" Bellman-Ford maintains two distance labels—`oldDist` and `newDist`—on each node. In each sweep, a pull-style operator is applied to an active node to read the `oldDist` values from its neighbors and update the `newDist` label of that active node. The roles of `oldDist` and `newDist` are reversed in the next iteration.

The computations performed by each sweep in Jacobi-style Bellman-Ford can be expressed succinctly in terms of a generalized matrix-vector product, where the matrix is the incidence matrix of the graph, and the vector gathers node labels—either `oldDist` or `newDist`—into a single data structure. Figure 3 shows a directed graph and its incidence matrix *A*. Each node has labels named *x* and *y*, and these labels can be collected to form the vectors *x* and *y*. If there is an edge (*i*,*j*) in the graph, *j* is said to be an "out-neighbor" of *i*, and *i* is said to be an "in-neighbor" of *j*. The matrix computation $y = y + Ax$ can be interpreted graphically as a topology-driven computation in which for each node *i* and for each of its out-neighbors *j*, the product of the weight on edge (*i*,*j*) and the value of the *x* label of *j* is added to the *y* label of *i*. The matrix computation $y = y + A^T\underline{x}$ with the transpose of the incidence matrix represents a similar computation with the in-neighbors of each node *i*.

Standard linear algebra uses arithmetic operations over the field $(\mathbb{R},+,x)$. The generalized matrix-vector product for the SSSP problem uses operations over the algebraic semi-ring $(\mathbb{R},\min,+)$ and computes $A^T\underline{x}$.

c This is a version of the delta-stepping algorithm of Meyer et al.[25] that does not require preprocessing; PHAST is another SSSP algorithm for high-diameter graphs (such as road networks) that quickly preprocess the graph to answer SSSP queries.[6]

d Strictly speaking, this is true if all edge labels are approximately equal in value.

Collaborative filtering. Given an incomplete database of ratings given by users to items, the goal is to predict how these users would rate items they have not yet rated. One way to solve this problem is through low-rank approximation. The database of ratings is represented as a matrix R, and low-rank approximation tries to find two low-dimensional dense matrices W and H such that $R \approx WH$, as shown in Figure 4.

Low-rank approximation can be formulated as a graph problem. The database is represented as a bipartite graph between users and items; if user u gave a rating r to an item v, there is an edge (u,v) in the graph with weight r. The matrices W and H are represented by vectors of length k associated with the nodes representing users and items, respectively, as shown in Figure 4. One way to compute these vectors is to use an iterative algorithm like stochastic gradient descent SGD, which initializes these vectors to some values and then makes a number of sweeps over the bipartite graph until a convergence test is satisfied. In each sweep, all edges (u,v) are visited in some order. If the inner product of the vectors on nodes u and v is not equal to the weight on edge (u,v), the difference is used to update the two vectors. This is a topology-driven algorithm in which edges are active.

Parallel Graph Analytics

Parallelism can be exploited by processing active nodes in parallel, subject to neighborhood and ordering constraints. Since neighborhoods can overlap, the memory model, which defines the semantics of reads and writes in overlapped regions, may prevent some activities with overlapping neighborhoods from executing in parallel. In addition, ordering constraints between activities must be enforced. We call this pattern of parallelism "amorphous" data-parallelism;[29] it is a generalization of data-parallelism in which there may be neighborhood and ordering constraints that prevent all activities from executing in parallel, and the execution of an activity may create new activities.

Figure 5 outlines the important choices in implementing parallel graph analytics programs. There are two popular memory models: bulk-synchronous parallel (BSP)-style semantics[37] and transactional semantics.

BSP-style semantics. The program is executed in rounds (also known as "super-steps"), with barrier synchronization between rounds. Writes to the graph are considered communication from a round to the following round, so are applied to the graph only at the beginning of the following round. Multiple updates to a label are resolved as in the parallel random access machine models (such as by using a reduction operation to combine the updates into a single update).[13] Each round thus consists of updating the graph based on values from the previous round, performing activities, and sending values to the next round.

As an example, consider the execution of the Bellman-Ford algorithm under BSP-style semantics. Each sweep over the graph is performed in one round. In each such round, the operator is applied to all nodes in parallel, but the updates to node labels are buffered and applied to the graph at the beginning of the next round. Multiple updates to the same node are combined by computing their minimum. Note under these semantics, the Bellman-Ford algorithm behaves like the Jacobi-style Bellman-Ford algorithm discussed earlier; updates made in a round become available to other relaxations only in the next round.

To reduce the overheads of data transfers between rounds, practical implementations of BSP-style semantics, particularly on distributed-memory machines, use a more coarse-grain approach in which tasks are associated not with individual activities but with logical or physical partitions of the graph. A task executes all activities whose active nodes lie in its partition; updates within the partition are performed in place and are available in

Figure 3. Graphical interpretation of generalized MVM.

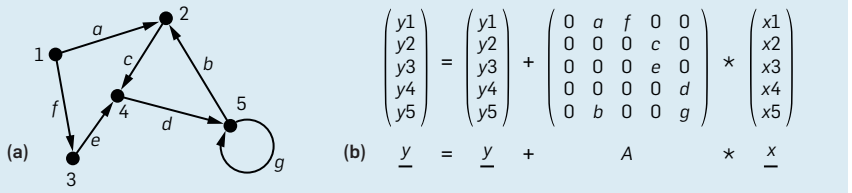


Figure 4. Collaborative filtering.

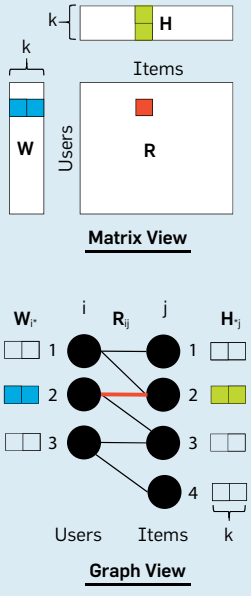
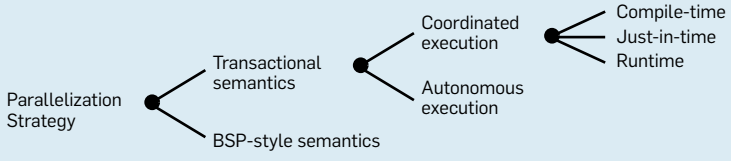


Figure 5. Parallelization strategies.



the same round to other activities, but updates to remote nodes are buffered and applied at the beginning of each super-step.

BSP-style parallelization may work well for graph analytics applications in which the number of activities in each round is large enough to keep the processors of the parallel machine busy; one example is breadth-first search (BFS) on a power-law graph. Each such round handles nodes at a single BFS level and computes labels for nodes at the next BFS level. Since the average diameter of power-law graphs is small, most rounds involve many parallel activities.

On the other hand, BSP-style parallelization may not perform well for graphs with high average diameter (such as road networks and meshes). This is because the number of super-steps required to execute the algorithm may be large, and the number of activities in each super-step may be small, as discussed later.

Transactional semantics. In this model, parallel execution of activities is required to produce the same answer as executing activities one at a time in some order that respects priorities. This means activities should not "see" concurrently executing activities, and the updates made by an activity should become visible to other activities only after that activity completes execution. Formally, these two properties of transactional execution are known as "isolation" and "atomicity."[11]

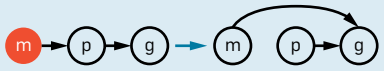
Transactional semantics are implemented by preventing activities from executing in parallel if they conflict. For unordered algorithms, a conservative definition is that activities conflict if their neighborhoods overlap. In Figure 1, activities *A* and *B* conflict because node *n* is in both their neighborhoods. Activities *C* and *D* do not conflict and can be executed in parallel with *A* or *B*. It is possible to use more relaxed definitions like commutativity checking, which exploits the fact that many graph analytics algorithms have commutative updates that can be implemented through atomics, thus eliding full transactional semantics.[18]

Given a definition of conflicts, the implementation must ensure conflicting activities do not update the graph in parallel. This can be accomplished through "autonomous scheduling" or "coordinated scheduling."

In autonomous scheduling, activities are executed speculatively.[31] If a conflict is detected with other concurrently executing activities, some activities are aborted, enabling others to make progress; otherwise, the activity commits, and its updates become visible to other activities.

Coordinated scheduling strategies ensure conflicting activities do not execute simultaneously.[29] For some algorithms (such as those that can be expressed through a generalized sparse matrix-vector product), static analysis of the operator shows active nodes can be executed in parallel without any conflict checking. This is called "static parallelization" and is similar to auto-parallelization of dense array programs. Just-in-time parallelization preprocesses the input graph to find conflict-free schedules (such as through graph coloring); this technique can be used for topology-driven algorithms like Bellman-Ford in which neighborhoods are independent of data values. Runtime parallelization is general; the algorithm is executed in a series of rounds, and in each round, a set of active nodes is chosen, their neighborhoods are computed, and a set of non-conflicting activities are selected and executed. This approach has been used for deterministic execution of unordered algorithms.[28]

Figure 6. With pointer-jumping, *m*'s parent pointer is redirected from parent *p* to grandparent *g*.

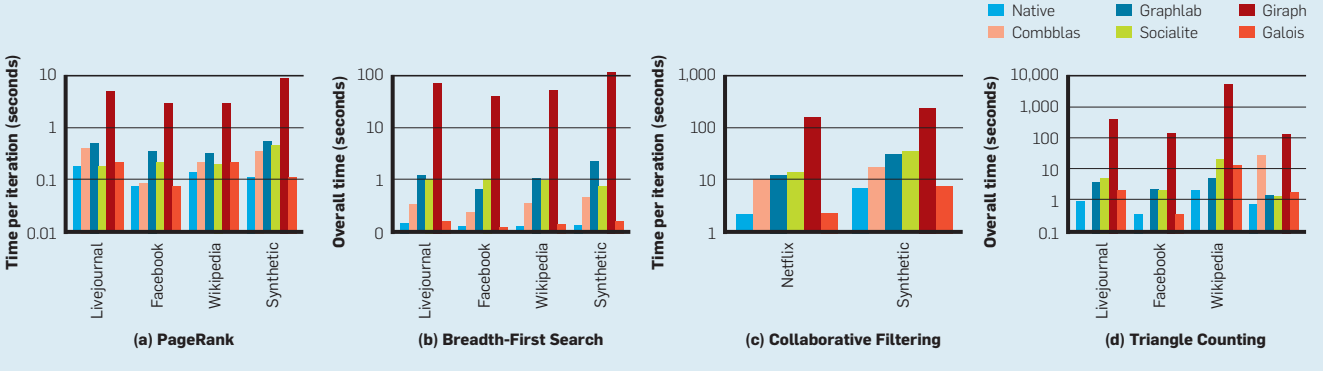


Systems for Graph Analytics

Systems for graph analytics usually support a subset of the algorithm classes and parallelization strategies discussed earlier. Here, we restrict ourselves to describing only a few of the many interesting systems for parallel graph analytics in the literature.

Most systems support only "vertex programs," an abstraction introduced by the Pregel system;[23] in the operator formulation, vertex programs are local computation operators in which the neighborhood is restricted to be the active node and its immediate neighbors. Vertex programs have limited expressiveness because important graph operations like "pointer-jumping," as in Figure 6, cannot be expressed as vertex programs. Efficient parallel algorithms for problems like connected components use extended versions of pointer-jumping that are similar to the `find` operation in disjoint-set union-find

Figure 7. Comparison of native implementations with graph analytics frameworks using shared-memory from Satish et al.[32]



in `disjoint-set union-find`.[13,14] Programming systems that do not support these types of operators may require inefficient algorithms for some problems.

Since very large graph datasets may not fit into the memory of a single host, one solution is to partition the graph among the hosts of a distributed-memory cluster. Some systems partition graph nodes, while others partition graph edges. A node partition, also known as a "1-D partition," assigns each node to one host; most graph partitioners try to partition nodes more or less equally among hosts while minimizing the number of edges that span multiple hosts. This works well for topology-driven algorithms on uniform-degree graphs since all nodes are active and the amount of computation is roughly the same at all nodes. However, node partitioning may lead to computation and communication imbalance for power-law graphs due to the lumpy nature of node degrees in these graphs. One solution is to partition edges among hosts, an approach sometimes called "2-D partitioning" because it is analogous to partitioning the matrix representation of the graph into 2D blocks and assigning each block to a host.

Examples of distributed-memory graph analytics systems include the following:

CombBLAS. CombBLAS[4] pioneered the generalized sparse matrix operations approach to formulating graph algorithms and is well suited for unordered, topology-driven algorithms with pull-style operators like Jacobi-style Bellman-Ford; CombBLAS supports 2-D partitions;

Pregel. Pregel[23] introduced the concept of vertex programs and supports a limited form of morph operators; Pregel's parallelization strategy uses BSP-style semantics and supports 1-D partitions;

Giraph. Giraph[8] is an implementation of Pregel from Facebook and implemented on top of the Hadoop platform for fault tolerance; and

PowerGraph. PowerGraph[9] is a distributed-memory programming model for vertex programs with BSP-style semantics and supports an implementation of 2D partitioning called "vertex cuts"; GraphLab[20] is an earlier shared-memory version of PowerGraph.

Examples of shared-memory systems include the following:

Ligra. Ligra[33] is a shared-memory programming model for vertex programs with BSP semantics and can switch automatically between push-style and pull-style operators during execution of an algorithm, which is useful for some algorithms on power-law graphs; and

Galois. Galois[27] is a general shared-memory programming model, implemented in sequential C++, and, unlike the systems outlined here, is not restricted to graph analytics; it supports morph operators and operators with arbitrary neighborhoods and implements transactional semantics with both autonomous and coordinated runtime scheduling.

Performance Studies

Graph analytics systems make the job of the application programmer easier through abstraction but may also impose limitations; for example, the best algorithm for a problem may not be expressible in the abstractions supported by a given system. Moreover, abstraction may introduce a performance penalty compared to explicitly parallel constructs like pthreads and MPI. This section describes performance studies that shed light on these matters.

Power-law graphs. A 2014 Intel study[32] considered four applications: PageRank, BFS, triangle counting, and collaborative filtering. The researchers wrote programs for each application in several frameworks, including CombBLAS, Graphlab, Giraph, and Galois. As a baseline for performance comparisons, the researchers also wrote parallel programs for each application in a "native" parallel notation—either pthreads or OpenMP.

We described BFS and collaborative filtering earlier in this article. PageRank is a topology-driven algorithm with a push-style operator on directed graphs, computing the relative importance of nodes in a graph as a function of its incoming neighbors. Triangle counting is a simple example of finding

Figure 8. Comparison of native implementations with graph analytics frameworks using distributed-memory from Satish et al.[32]

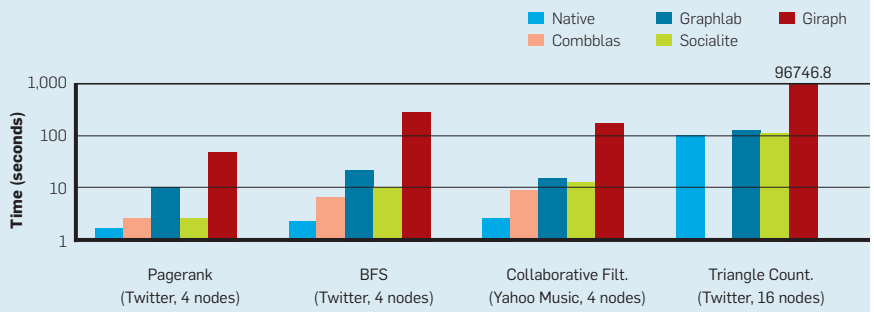
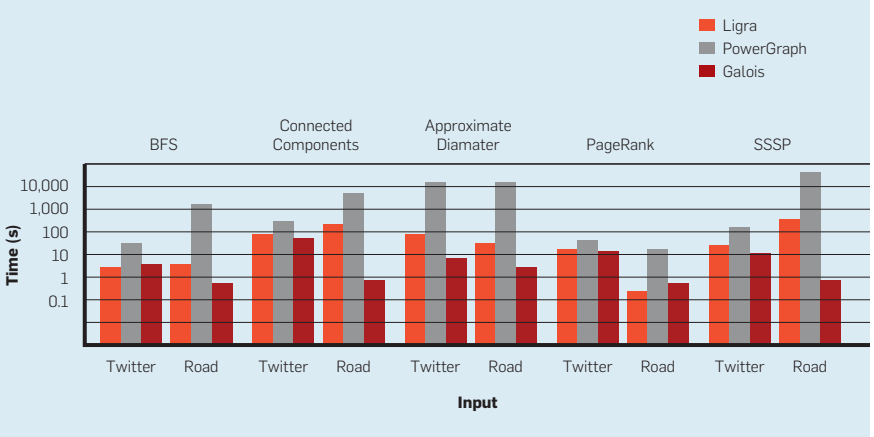


Figure 9. Comparison of graph analytics frameworks from Nguyen et al.[27]



motifs in graphs. A triangle is formed in an undirected graph whenever two nodes have a common neighbor. The problem is to count the number of triangles in a given undirected graph.

Figure 7 shows the running times of these programs on several power-law graphs on a 24-core Intel Xeon CPU E5-2697 with 64GB of DRAM. Note the *y*-axis is a log-scale. The Facebook graph has three million nodes and 42 million edges. The synthetic graph has 537 million nodes and 8.6 billion edges. For collaborative filtering, a synthetic bipartite graph included 63 million users, 1.3 million items, and 17 billion ratings. PageRank times are for one iteration.

While the native parallel programs performed best, Galois programs are close in performance; the Intel study reported they are slower by a factor of 1.1–1.2 for PageRank, BFS, and collaborative filtering and 2.5X for triangle counting. Giraph is 100–1,000 times slower than native code due to I/O in the Hadoop system. While PageRank and BFS fit the generalized sparse matrix vector multiply pattern directly and are thus easy to express in CombBLAS, triangle counting is more tricky. The implementation of triangle counting in the Intel study[32] computed the matrix product A^2 and ran out of memory; Azad et al.[1] described a better CombBLAS algorithm.

Figure 8 shows performance numbers from the Intel study[32] for a four-host distributed-memory system; there are no performance bars for Galois, since it is shared-memory, although a distributed-memory version will be available in 2016. In this experiment, native parallel programs were written with a combination of MPI and OpenMP/pthreads. As in the shared-memory case, Giraph is substantially slower than the other systems due to I/O. CombBLAS and Socialite perform comparably and are faster than GraphLab, which is limited by network bottlenecks.[32]

Influence of graph structure. Nguyen et al.[27] studied the interaction between graph structure and parallelization strategy. They evaluated five problems in PowerGraph, Ligra, and Galois—BFS, connected components, approximate diameter, PageRank, and SSSP. Their experiments measured running times on a 40-core Intel E7-4860 with 128GB of memory for two inputs graphs—a Twitter graph (51 million nodes, 2.0 billion edges) and a U.S. road network (24 million nodes, 58 million edges). The Twitter graph is a power-law graph, whereas the road network is a uniform-degree graph smaller than the Twitter graph but with a much larger average diameter. Figure 9 shows these running times; note the *y*-axis is a log-scale.

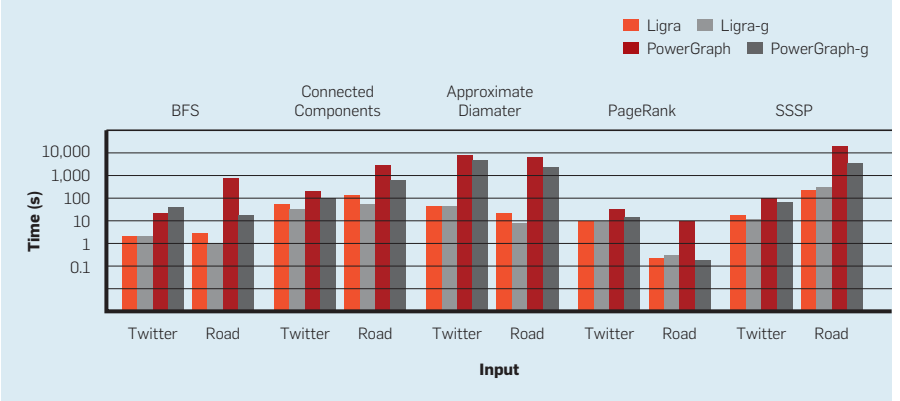
In the BSP-style frameworks—Ligra and PowerGraph—BFS and SSSP take longer on the road network than on the Twitter graph, even though the road network is almost two orders of magnitude smaller than the Twitter graph. The reason for the difference is BSP-style parallelization does not expose much parallelism if the number of activities in each round is small and the number of rounds is large, as is the case with road networks and other uniform-degree graphs. For SSSP, Ligra, and PowerGraph, implementations use the Bellman-Ford algorithm but filter nodes so each sweep considers only those nodes with labels updated by the previous sweep. In contrast, the Galois implementations can use data-driven algorithms like asynchronous delta-stepping, and the asynchronous, priority-guided execution of activities in the Galois system adapts to both the power-law graph and the road network.

The performance numbers for connected components highlight the limitations of vertex programs. The Galois implementation uses an efficient algorithm based on pointer-jumping that cannot be expressed in frameworks limited to vertex programs, as described earlier. Ligra and PowerGraph use a label-propagation algorithm; nodes are given integer labels, and the node with the smallest label in a component propagates its label to all other nodes in that component. When the graph diameter is small, as it is for the Twitter graph, label propagation works fine, but for the road network, it is two or three orders of magnitude slower than the asynchronous pointer-jumping algorithm in Galois.

Overall, there is a wide variation in running times across different programming models solving the same problem. The variation is least for PageRank, which uses a topology-driven algorithm in all these systems. For the other problems, the performance differences between programming models can be several orders of magnitude and are quite stark for the road input, for which the large diameter heavily penalizes BSP-style parallelization and favors asynchronous, data-driven execution of activities.

Since the programming models supported by graph DSLs (such as PowerGraph and Ligra) are restrictions of the Galois programming model, they can be implemented on top of Galois through code adaptors consisting of a few hundred lines of C++ code; we call these implementations "PowerGraph-g" and "Ligra-g." Figure 10 shows the performance of these implementations is approximately the same as the performance of the native PowerGraph and Ligra systems, showing Galois can be considered a graph analytics engine

Figure 10. Comparison of graph analytics frameworks rewritten on top of Galois from Nguyen et al.[27]



on top of which custom graph DSLs can be layered.

Research Directions

Ongoing research in parallel graph analytics spans the gamut, from new domain-specific languages[12,30] to novel processor architectures for supporting graph analytics[2,16,22] and is summarized in the online appendix.

Over the past decade, the parallel computing research community has acquired a reasonably good understanding of the landscape of parallel graph analytics algorithms. Exploiting these insights to build efficiently engineered systems for parallel graph analytics, particularly on nontraditional architectures, and incorporating graph analytics into the larger ecosystem of parallel computing are the next major challenges for this new and rapidly growing research area.

Acknowledgments

We thank Jon Kleinberg (Cornell University) and Vijaya Ramachandran and Amber Hassaan (the University of Texas at Austin) for assistance with this article and the members of the Intelligent Software Systems group at the University of Texas at Austin for their contributions to the research it describes.

References

1. Azad, A., Buluc, A., and Gilbert, J.R. Parallel triangle counting and enumeration using matrix algebra. In *Proceedings of the 29th International Parallel & Distributed Processing Symposium Workshop* (Hyderabad, India, May 25–29). IEEE International, 2015, 804–811.
2. Bader, D.A. and Madduri, K. Designing multithreaded algorithms for breadth-first search and st-connectivity on the Cray MTA-2. In *Proceedings of the 2006 International Conference on Parallel Processing* (Columbus, OH, Aug. 14–18). IEEE Computer Society Press, 2006, 523–530.
3. Bertsekas, D.P. and Tsitsiklis, J.N. *Parallel and Distributed Computation: Numerical Methods.* Prentice-Hall, Inc., Upper Saddle River, NJ, 1989.
4. Buluç, A. and Gilbert, J.R. The combinatorial BLAS: Design, implementation, and applications. *International Journal of High Performance Computing Applications 25*, 4 (Nov. 2011), 496–509.
5. Cormen, T., Leiserson, C., Rivest, R., and Stein, C., Eds. *Introduction to Algorithms.* MIT Press, Cambridge, MA, 2001.
6. Delling, D., Goldberg, A.V., Nowatzyk, A., and Werneck, R.F. Phast: Hardware-accelerated shortest path trees. In *Proceedings of the 25th International Parallel and Distributed Processing Symposium* (Anchorage, AK, May 16–20). IEEE Computer Society Press, 2011, 921–931.
7. Fernholz, D. and Ramachandran, V. The diameter of sparse random graphs. *Random Structures & Algorithms 31*, 4 (Dec. 2007), 482–516.
8. Giraph; http://giraph.apache.org/
9. Gonzalez, J.E., Low, Y., Gu, H., Bickson, D., and Guestrin, C. PowerGraph: Distributed graph-parallel computation on natural graphs. In *Proceedings of the USENIX Conference on Operating Systems Design and Implementation* (Hollywood, CA, Oct. 8–10). USENIX Association, Berkeley, CA, 2012, 17–30.
10. Harish, P. and Narayanan, P.J. Accelerating large graph algorithms on the GPU using CUDA. In *Proceedings of the 14th International Conference on High Performance Computing* (Goa, India, Dec. 19–22). Springer-Verlag, Berlin, Heidelberg, 2007, 197–208.
11. Herlihy, M. and Shavit, N. *The Art of Multiprocessor Programming.* Morgan-Kaufmann, San Francisco, CA, 2012.
12. Hong, S., Chafi, H., Sedlar, E., and Olukotun, K. Green-Marl: A DSL for easy and efficient graph analysis. In *Proceedings of the 17th International Conference on Architectural Support for Programming Languages and Operating Systems* (London, U.K., Mar. 3–7). ACM Press, New York, 2012, 349–362.
13. JaJa, J. *An Introduction to Parallel Algorithms.* Addison-Wesley, Reading, MA, 1992.
14. Karp, R.M. and Ramachandran, V. *A Survey of Parallel Algorithms for Shared-Memory Machines, Technical Report UCB/CSD-88-408.* EECS Department, University of California, Berkeley, Mar. 1988; http://www.eecs.berkeley.edu/Pubs/TechRpts/1988/CSD-88-408.pdf
15. Khorasani, K., Vora, K., Gupta, R., and Bhuyan, L.N. Cusha: Vertex-centric graph processing on GPUs. In *Proceedings of the 23rd International Symposium on High-performance Parallel and Distributed Computing* (Vancouver, Canada, June 23–27). ACM Press, New York, 2014, 239–252.
16. Kim, J. and Batten, C. Accelerating irregular algorithms on GPGPUs using fine-grain hardware worklists. In *Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture* (Cambridge, U.K., Dec. 13–17). IEEE Computer Society Press, 2014, 75–87.
17. Kulkarni, M., Burtscher, M., Casçaval, C., and Pingali, K. Lonestar: A suite of parallel irregular programs. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software* (Boston, MA, Apr. 26–28). IEEE Computer Society Press, 2009; http://iss.ices.utexas.edu/?p=projects/galois/lonestar
18. Kulkarni, M., Nguyen, D., Prountzos, D., Sui, X., and Pingali, K. Exploiting the commutativity lattice. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation* (San Jose, CA, June 4–8). ACM Press, New York, 2011, 542–555.
19. Kumar, R., Raghavan, P., Rajagopalan, S., Sivakumar, D., Tompkins, A., and Upfal, E. The Web as a graph. In *Proceedings of the 19th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems* (Dallas, TX, May 15–17). ACM Press, New York, 2000, 1–10.
20. Low, Y., Gonzalez, J., Kyrola, A., Bickson, D., Guestrin, C., and Hellerstein, J.M. GraphLab: A new parallel framework for machine learning. In *Proceedings of the 26th Conference on Uncertainty in Artificial Intelligence* (Catalina Island, CA, July 8–11). Association for Uncertainty in Artificial Intelligence, 2010.
21. Lu, L. The diameter of random massive graphs. In *Proceedings of the 12th Annual ACM-SIAM Symposium on Discrete Algorithms* (San Francisco, CA, Jan. 9–11). Society for Industrial and Applied Mathematics, Philadelphia, PA, 2001, 912–921.
22. Madduri, K., Feo, J., Cong, G., and Bader, D.A. Design of multithreaded algorithms for combinatorial problems. Chapter 31 in *Handbook of Parallel Computing: Models, Algorithms and Applications*, S. Rajasekaran and J. Reif, Eds. Chapman and Hall/CRC, Boca Raton, FL, 2007.
23. Malewicz, G., Austern, M.H., Bik, A.J., Dehnert, J.C., Horn, I., Leiser, N., and Czajkowski, G. Pregel: A system for large-scale graph processing. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data* (Indianapolis, IN, June 6–11). ACM Press, New York, 2010, 135–146.
24. Merrill, D., Garland, M., and Grimshaw, A. Scalable GPU graph traversal. *SIGPLAN Notices 47*, 8 (Feb. 2012), 117–128.
25. Meyer, U. and Sanders, P. Delta-stepping: A parallel single-source shortest-path algorithm. In *Proceedings of the Sixth Annual European Symposium on Algorithms* (Venice, Italy, Aug. 24–26). Springer-Verlag, London, U.K., 1998, 393–404.
26. Nasre, R., Burtscher, M., and Pingali, K. Morph algorithms on GPUs. In *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Shenzhen, China, Feb. 23–27). ACM Press, New York, 2013, 147–156.
27. Nguyen, D., Lenharth, A., and K. Pingali. A lightweight infrastructure for graph analytics. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles* (Farmington, PA, Nov. 3–6). ACM Press, New York, 2013, 456–471.
28. Nguyen, D., Lenharth, A., and Pingali, K. Deterministic Galois: On-demand, portable and parameterless. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems* (Salt Lake City, UT, Mar. 1–5). ACM Press, New York, 2014, 499–512.
29. Pingali, K., Nguyen, D., Kulkarni, M., Burtscher, M., Hassaan, M.A., Kaleem, R., Lee, T.-H., Lenharth, A., Manevich, R., Méndez-Lojo, M., Prountzos, D., and Sui, X. The Tao of parallelism in algorithms. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation* (San Jose, CA, June 4–8). ACM Press, New York, 2011, 12–25.
30. Prountzos, D., Manevich, R., and Pingali, K. Synthesizing parallel graph programs via automated planning. In *Proceedings of the 36th Annual SIGPLAN Conference on Programming Language Design and Implementation* (Portland, OR, June 13–17). ACM Press, New York, 2015, 533–544.
31. Rauchwerger, L. and Padua, D.A. The LRPD test: Speculative runtime parallelization of loops with privatization and reduction parallelization. *IEEE Transactions on Parallel Distributed Systems 10*, 2 (Feb. 1999), 160–180.
32. Satish, N., Sundaram, N., Patwary, M.M.A., Seo, J., Park, J., Hassaan, M.A., Sengupta, S., Yin, Z., and Dubey, P. Navigating the maze of graph analytics frameworks using massive graph datasets. In *Proceedings of the ACM SIGMOD International Conference on Management of Data* (Snowbird, UT, June 22–27). ACM Press, New York, 2014, 979–990.
33. Shun, J. and Blelloch, G.E. Ligra: A lightweight graph-processing framework for shared memory. In *Proceedings of the ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Shenzhen, China, Feb. 23–27). ACM Press, New York, 2013, 135–146.
34. Shun, J., Blelloch, G.E., Fineman, J.T., Gibbons, P.B., Kyrola, A., Simhadri, H.V., and Tangwongsan, K. Brief announcement: The problem-based benchmark suite. In *Proceedings of the 24th Annual ACM Symposium on Parallelism in Algorithms and Architectures* (Pittsburgh, PA, June 25–27). ACM Press, New York, 2012, 68–70.
35. Siek, J., Lee, L.-Q., and Lumsdaine, A. *The Boost Graph Library: User Guide and Reference Manual.* Addison-Wesley, Reading, MA, 2001.
36. Ugander, J., Karrer, B., Backstrom, L., and Marlow, C. *The Anatomy of the Facebook Social Graph.* Technical Report, 2011; http://arxiv.org/abs/1111.4503
37. Valiant, L.G. A bridging model for parallel computation. *Commun. ACM 33*, 8 (Aug. 1990), 103–111.
38. Xin, R.S., Gonzalez, J.E., Franklin, M.J., and Stoica, I. GraphX: A resilient distributed graph system on Spark. In *Proceedings of the First International Workshop on Graph Data Management Experiences and Systems* (New York, June 22–27). ACM Press, New York, 2013.

Andrew Lenharth (lenharth@ices.utexas.edu) is a research associate in the Institute for Computational Engineering and Sciences and a lecturer in the Department of Computer Science at the University of Texas at Austin.

Donald Nguyen (ddn@cs.utexas.edu) is a senior developer in Synthace Ltd., London, U.K.; he received his Ph.D. from the University of Texas at Austin in 2015 while doing research for this article.

Keshav Pingali (pingali@cs.utexas.edu) is the W.A. "Tex" Moncrief Chair of Grid and Distributed Computing, a professor in the Department of Computer Science, and a professor in the Institute for Computational Engineering and Sciences at the University of Texas at Austin.

© 2016 ACM 0001-0782/16/05 $15.00

Watch the authors discuss their work in this exclusive *Communications* video. http://cacm.acm.org/videos/parallel-graph-analytics